Lecture 02: Boolean logic and transistors

Goals

- describe the three basic Boolean operations, plus an extra
- produce truth tables for arbitrary Boolean expressions
- describe the behavior of PMOS and NMOS transistors
- use transistors to implement basic Boolean operations
- define *composition*

verify waitlist

quiz Friday, will talk about what to expect and how to prepare

problems out soon, will be due 1 week from Monday, will talk on Friday about the process there (want yo

last time, I showed a real circuit and then the equivalent in Logisim

pointed out that we lost a lot of detail

- no notion of voltage, just `on' or`off'
- no resistor
- no connection to ground
- no circular path for the current to flow around

let us now officially set aside such silly details in the name of abstraction

let us now hold that a wire can be either on or off, high or low, one or zero

or `floating'' (ie, not connected to either a zero or a one), sometimes represented as`Z" or ``U" (

what can we do with ones and zeroes?

George Boole

in 1847, George Boole published a pamplet "The Mathematical Analysis of Logic"

in which he proposed a set of operations we can perform on only two values

sometimes known as False and True, or zero and one (or low and high)

only until decades after his death did this become known as Boolean Algebra

he proposed three basic operations:

- negation/inversion (NOT)
- conjunction (AND)
- disjunction (OR)

a *truth table* (which you may have seen before) is an enumeration of all possible inputs and their corresp
operation

the truth table for NOT looks like so:

```
  A   | not A
--------------
```

```
  0   |   1
  1   |   0
```

the truth table for AND looks like so:

```
A | B | A AND B
---------------
0 | 0 |    0
0 | 1 |    0
1 | 0 |    0
1 | 1 |    1
```

the truth table for OR looks like so:

```
A | B | A OR B
--------------
0 | 0 |   0
0 | 1 |   1
1 | 0 |   1
1 | 1 |   1
```

there exist symbols for these operations (which I will draw on the board) and you should be aware they e
worry about them

you may also see truth tables written with T (or true) instead of a 1, and F (or false) instead of a 0

these are indeed equivalent, and I will accept both on quizzes and exams

but since the circuit stuff prefers 0 and 1, that's what I'll usually use in truth tables

these operations can be *composed*, meaning "combined together"

composition is the act of combining together smaller things to produce bigger ones

I could use both OR and AND to form the expression "A AND B OR C"

the difficulty here is that it's difficult to tell how to evaluate this expression: do we consider the AND first c

therefore, I will always (and I encourage you to always) use parentheses to make it explicit

to indicate that the OR in the above expression should be performed first, write the expression as "A ANI

here's a slightly more complicated example:

(A OR B) AND NOT (A AND B)

we could create a truth table for it by again enumerating every combination of values for A and B and eva

when you have a more complicated expression like this, one way to evaluate it is to do the following:

- pick values for the variables (we can start with A is false and B is false)
- replace each variable with its chosen value (which, for this example, would give "(false OR false) ANI
- find a part of the expression that we can immediately resolve according to the rules of AND/OR/NOT
  (false OR false)" could become just "false", giving a new expression "false AND NOT (false AND false
- and keep repeating this process until we are left with a single value, which is the value of the entire e

we can iterate through all combinations of values for the variables to give us a truth table:

```
A | B | (A OR B) AND NOT (A AND B)
------------------------------
0 | 0 |             0
```

```
0 | 1 |             1
1 | 0 |             1
1 | 1 |             0
```

this particular operation is actually pretty widely-used, so it is given a special name and even a special sy
called XOR

the idea is that it means "one or the other but not both"

these are very simple operations

it's not at all clear how they help us build a computer

but they will

so how do we make implement these operations with wires?

first: digression to some other electrical components

a SWITCH controls current: closed, current flows; open, current doesn't

think of a light switch: when it's up, the current flows and the lights are on; when it's down, current doesn

(draw open circuit on board, with floating output; and closed circuit)

a RELAY is an electronically-controlled switch

so instead of manually flipping the (light) switch, you apply current

(modify drawing on board)

the venerable vacuum tube does the same thing

replica of first transistor

John Bardeen, Walter Brattain, William Shockley of Bell Labs invented the transistor in the late 1940s

and in 1956 won the Nobel Prize for it

crazy efficient: size, electrical properties, manufacturing, durability

they are, fundamentally, precisely what make computers "work" to this day

three connections: source, drain, and gate

two types: pmos and nmos

positive/negative metal-oxide semiconductor

let's tackle the NMOS first

it passes through a 0 when gate is 1; otherwise drain is disconnected

we could imagine a truth table like so:

```
gate | source | drain
---------------------
 0   |   0    |  U
 0   |   1    |  U
 1   |   0    |  0
 1   |   1    |  U
```

(here, U represents "unknown"—if "drain" is disconnected, it isn't being given either a zero or a one… by

the PMOS works in the opposite way: it passes through a 1 when gate is 0

its truth table would look thusly:

gate	source	drain
0	0	U
0	1	1
1	0	U
1	1	U

(note that these two tables abuse the truth table mechanic a bit, in that "U" isn't technically a truth value. gets the job done)

(note also that these don't *really* behave like the implication of the switch or relay above: the implication t input is passed to the output when a switch is closed. instead, a transistor is more selective: each type o value when activated)

transistor_demo.circ

we can use the poke tool to change the value of the input

change it to an nmos and see the behavior change

in Logisim, you tell the difference between PMOS and NMOS because the PMOS transistor has a little c letter "P")

you can also tell which end is the source and which end is the drain because the little arrow points from s

now let's try to implement the inversion operation using transistors

let's go one row of the truth table at a time: start by considering the case when the input is 0 and we wan

will either of the transistors output a 1?

yes! the pmos transistor will output a 1, but only when its gate is 0 and its source is 1

we've got a 0 (because A is 0) but we need to provide a 1

so we connect A to the gate input of the PMOS transistor

and we place and connect a power component to its source input

now, in Logisim, we can see that, when A is 0, the transistor does indeed output a 1

when A is 1, the transistor output is undefined, but that's okay because we haven't considered that case

let's consider it now: when the input is 1, we want the output to be 0

looking at the truth tables for both transistors, we see that the NMOS will output a 0, but only when gate

A will be one, so we can place an NMOS transistor and connect A to its gate

and we have to place a Ground component to get a 0, which we connect to the NMOS transistor's sourc

and we can confirm in Logisim that the drain of this transistor is indeed 0 when A is 1

so now we've got two transistors:

when A is 0, one first transistor outputs a 1 and the second outputs a U

when A is 1, the second transistor outputs a 1 and the first outputs a U

and here's the trick: if we connect those two outputs together, they will never disagree because the U ou
zero value, just not a value at all), so there's nothing to cause conflict

so we connect those two outputs together and that is the output of our entire circuit

you can see the result here: inverter.circ

how does this even work?

disconnect the drains

show that when one is driven, the other is always floating

what happens if a wire is connected to both 0 and 1 at the same time?

short circuit

red wire and error in Logisim

when you make circuits, the final versions should *never* have red wires

nor should it have outputs which are ever undefined

how can we convince ourselves this works?

test plan

in this case, enumerate all inputs and desired outputs, verify them

(coming up with a way to test your work will be a continuing theme)

nor_gate.circ

more complex example: the NOR operation (that is, not-or)

come up with a test plan

demonstrate it works

now: how to make a circuit that performs OR?

yep: copypasta the nor-gate and the inverter and combine the two

test it

we can compose circuits just like we composed the symbolic boolean expressions earlier

which lies at the heart of abstraction and, therefore, underlies all of CS

next time: more complex operations on ones and zeroes, abstract away transistors and get *logic gates*

Wikipedia article on transistor count has a table at the bottom showing numbers of transistors required tc

how to prepare for quiz

install Logisim-Evolution (instructions on the course website)

memorize the definitions (abstraction and composition)

run the practice problems on weathertop

to see a list:

```
$ 202 practice
```

to run a problem:

```
$ 202 practice t1p1m01
```

the questions on the quiz will be of exactly the same form as the questions in the practice problems

this is not intended to be surprising or tricky; it's intended to get you efficient effective with basic skills tha complicated things well

Definitions

The following definitions introduced in this lecture are fair-game for future quizzes. You will be expected t provided in these lecture notes.

- composition

Mechanical Skills

The following mechanical skills introduced in this lecture are fair-game for future quizzes. You may acces exactly resemble the questions on the quizzes) on weathertop.

```
t1p1m01    Evaluate Boolean expressions
t1p1m02    Evaluate circuit (transistors)
```

Last modified: 09/16/2026 12:45:01